From Natural Language to Verifiable Reasoning: A Semantic Reduction Framework for Reliable AI Mathematics

Yiran Lu

September 2026

Abstract

Large language models now produce fluent and frequently correct solutions to competition-level mathematics, yet the field lacks a shared account of what it means for such a solution to be *reliable*. We argue that the three dominant remedies — search guided by a verifier, formalisation inside a proof assistant, and reproducible benchmarking — all presuppose a step that is seldom made explicit and almost never measured: the reduction of an informal, untyped natural-language argument into a typed object that a verifier can accept. We call this the *semantic reduction layer*. We formalise a three-layer model (generation, reduction, verification), prove that verifier soundness does not transfer to the original claim when the reduction is unfaithful (Proposition 1), and bound end-to-end certified reliability by the weakest of the three layers (Proposition 2). We then contribute two operational artefacts the literature currently lacks: a five-level *Reliability Ladder* (L0–L4) for grading any AI mathematical output, and a twelve-item *Reduction Verification Checklist*. Re-reading published results through the ladder exposes a systematic *tier gap*: headline numbers are usually evidenced at L0–L2 while the informal reading of the same papers implies L3–L4. A fully worked (constructed) case study on an elementary algebraic identity exhibits the gap, and we pre-specify — but do not execute — a protocol for measuring reduction fidelity. We report no new empirical results; the contribution is conceptual, and we state its boundaries explicitly.

Keywords: AI for mathematics, reliability, autoformalisation, semantic reduction, proof assistants, verifiable reasoning.

1 Introduction

Mathematical reasoning has become one of the most visible testing grounds for large language models. Models trained on technical corpora reach strong performance on competition benchmarks without external tools [10, 7, 3]; neuro-symbolic systems solve olympiad geometry at a level approaching an average IMO gold medallist [15]; and reinforcement-learning agents operating inside the Lean prover have reached silver-medal-equivalent performance at the 2024 IMO [8]. Program-search methods have even produced new constructions for open problems in extremal combinatorics [12].

Alongside these results runs a quieter and more stubborn observation. Language models are generative devices trained to produce *plausible* text, and plausibility is not validity: they routinely emit confident, well-formed, and wrong content [9]. In mathematics this has an especially sharp consequence. A derivation can be locally fluent at every step, arrive at the correct final answer, and still be invalid, because answer-level grading does not read the reasoning [17, 11]. The mathematics community's answer to this has been, roughly, threefold: *search and verify*, *formalise*, and *evaluate reproducibly*.

This paper is about a step that all three presuppose and that none of them measures. To verify anything, one needs a *handle*: an object that a checker can act on. Natural language

offers no such object. Somewhere between the generated prose and the verifier, the argument must be *reduced* to a typed, semantically committed representation. We call this the *semantic reduction layer*, and we argue that it — not generation and not verification — is where the reliability of an AI mathematical pipeline is typically lost.

Why this matters now. The formal route is often presented as if it settled the question: if a kernel accepts a proof, the theorem is true. But a kernel accepts a *formal object*, and the relationship between that object and the informal claim a user actually asked about is established by the reduction, not by the kernel. When that relationship is loose, a green checkmark certifies something other than what the reader believes it certifies. This is not a hypothetical worry: at IMO 2024 the competition problems were first translated into formal mathematical language by humans before the proving systems were applied [8], and the largest reported formal-mathematics efforts lean on autoformalisation at massive scale precisely because this translation is hard [8, 1].

Contributions. We make five contributions, three conceptual and two operational:

(1) A *three-layer model* of an AI mathematical pipeline — generation, semantic reduction, verification — with explicitly typed interfaces between the layers (Section 3).

(2) Two *formal statements* about where reliability is lost: Proposition 1 (verifier soundness does not transfer across an unfaithful reduction) and Proposition 2 (end-to-end certified reliability is bounded above by the weakest layer).

(3) A *taxonomy* of seven reduction failure modes, derived from the model and illustrated against published systems (Section 4).

(4) A five-level *Reliability Ladder* (L0–L4) and a twelve-item *Reduction Verification Checklist* that operationalise the framework for practitioners and reviewers (Section 5).

(5) A *re-analysis* of published results showing a systematic tier gap, a fully worked (constructed) case study that exhibits it, and a pre-specified protocol for measuring reduction fidelity (Sections 6–8).

Scope and honesty of claims. This is a conceptual paper. We report *no new empirical results*. The case study in Section 7 is constructed for exposition and is labelled as such; the protocol in Section 8 is pre-specified but not executed. Every numerical claim about the literature is attributed to a cited source and was re-checked against that source on 2026-09-25; the verification record is given in Appendix A.

2 Related Work

2.1 Informal neural mathematical reasoning

Step-by-step prompting made multi-step arithmetic and algebraic reasoning tractable for language models [19]. Dedicated benchmarks followed: GSM8K provides 8.5K grade-school word problems [3] and MATH provides 12,500 competition problems with full step-by-step solutions [7]. Minerva showed that continued training on technical content yields state-of-the-art performance on such benchmarks without external tools [10]. All of these evaluate *answers*, not *arguments*.

2.2 Verification, search and process supervision

An early and influential response was to train a second model to judge correctness and rerank sampled solutions [3]. Uesato et al. [17] separated outcome supervision (label the final answer) from process supervision (label each step) and showed the distinction is empirically real: they improved final-answer error from 16.8% to 12.7% while reducing reasoning error *among final-answer-correct solutions* from 14.0% to 3.4%. That second number is the one that matters for this paper — it is a direct measurement of the gap between "the answer is right" and "the reasoning is right". Scaling process supervision, Lightman et al. [11] released PRM800K and reported that a process-supervised reward model solves 78% of a representative MATH subset under large best-of-n reranking. Note what these verifiers are: learned models. They are scored, not proved.

2.3 Formal methods and autoformalisation

Interactive theorem provers provide the handle that natural language lacks. `Lean` 4 is both a prover and a programming language [5], backed by a large, community-maintained mathematical library [14]; the correspondence between propositions and types is the theoretical ground on which such systems stand [18]. Benchmarks have been built to measure machine proving: miniF2F offers 488 cross-system olympiad statements [22], ProofNet offers 371 undergraduate-level examples pairing a formal statement with a natural-language statement and proof [1], and PutnamBench offers 1692 hand-constructed formalisations of 640 Putnam theorems across Lean 4, Isabelle and Coq, on which established provers solve only a handful [16]. The retrieval bottleneck in premise selection — a reduction-layer problem, in our terms — was isolated and addressed by LeanDojo [21]. Recent provers combine language models with proof-assistant feedback and tree search [20]. The strongest results to date come from combining a learned prover with a formal environment [15, 8].

2.4 Neuro-symbolic and tool-augmented systems

A parallel line externalises the parts of reasoning that machines do well: self-supervised API use [13] and program-of-thought prompting [2] move computation out of the model and into an interpreter. FunSearch pairs a language model with a systematic evaluator and discovers new constructions for the cap set problem [12]; related work uses machine learning to guide mathematicians' intuition on open problems [4]. These systems are reliable *where a checker exists* and silent elsewhere.

2.5 Positioning

Prior work excels at *generating*, at *checking*, and at *benchmarking*. What is comparatively under-theorised is the interface: the map from informal argument to checkable object. Autoformalisation papers study it as a translation task [1]; prover papers treat it as preprocessing. We treat it as a first-class layer with its own failure modes and its own metrics, and we supply artefacts (ladder, checklist) designed to make the layer auditable in ordinary practice.

3 A Three-Layer Model

3.1 Objects and interfaces

Definition 1 (Pipeline objects)**.** *Let c be an informal mathematical* claim *stated in natural language. A* derivation *for c is a finite sequence $d = \langle s_1, \ldots, s_n \rangle$ of informal reasoning steps. A* formal target Φ *is a well-typed expression of a formal theory T. A* semantic reduction *is a partial map $\rho : d \mapsto \Phi$. A* verifier *is a partial computable map $\mathcal{V} : \Phi \mapsto \{0, 1\}$.*

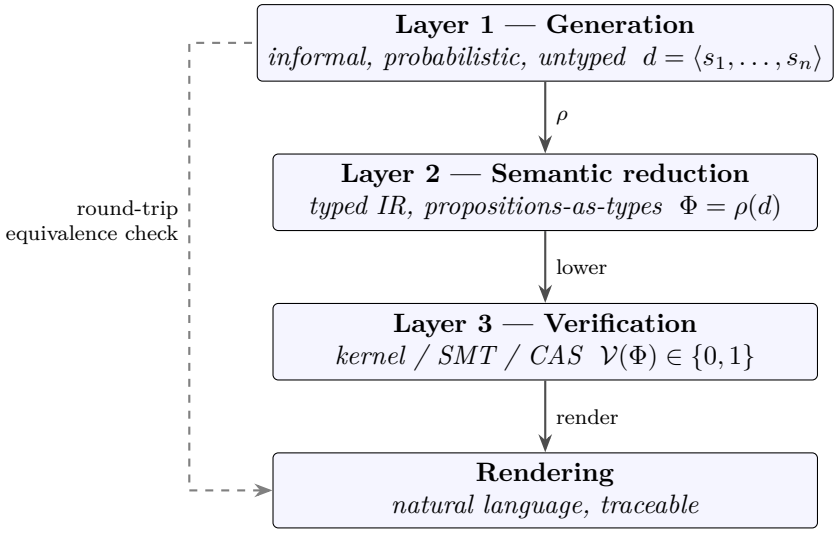


Figure 1: The three-layer architecture. Layer 2 is the object of study in this paper: it is the only layer whose failure is invisible to both of its neighbours.

The three layers are: **L1, generation** (produce d from c); **L2, semantic reduction** (produce $\Phi = \rho(d)$); **L3, verification** (evaluate $\mathcal{V}(\Phi)$). Natural language is the *interface* to the system; the typed representation is its *execution core*. Figure 1 shows the architecture, including the rendering and round-trip steps that make the reduction auditable.

3.2 Where reliability is lost

We use two standard properties, stated so that the gap is unambiguous.

Assumption 1 (Soundness)**.** *$\mathcal{V}$ is* sound *for T: $\mathcal{V}(\Phi) = 1 \implies T \vdash \Phi$.*

Definition 2 (Faithful reduction)**.** *Let $[\![\cdot]\!]$ denote semantic denotation. A reduction ρ is* faithful *for (c, d) iff $[\![\rho(d)]\!] = [\![c]\!]$. It is* sound-but-unfaithful *iff $\mathcal{V}(\rho(d)) = 1$ but $[\![\rho(d)]\!] \neq [\![c]\!]$.*

Proposition 1 (Reduction gap)**.** *Suppose $\mathcal{V}$ is sound for T. If ρ is not faithful for (c, d), then $\mathcal{V}(\rho(d)) = 1$ does* not *entail c in T.*

Proof. Soundness licenses exactly one inference: from $\mathcal{V}(\Phi) = 1$ to $T \vdash \Phi$, where $\Phi = \rho(d)$. It asserts nothing about the relation between Φ and c. By unfaithfulness, $[\![\Phi]\!] \neq [\![c]\!]$, so there exists a structure in which $T \vdash \Phi$ holds while c fails; in that structure the pipeline outputs a certificate while the user's claim is false. Hence no entailment from $\mathcal{V}(\Phi) = 1$ to c is available. □

Proposition 1 is almost embarrassingly simple, and that is the point: it is the formal version of a mistake that is easy to make in practice — reading "the proof checked" as "the claim I asked about is true".

Proposition 2 (Weakest-layer bound)**.** *Fix a distribution of problems and a pipeline. Let G, R and V be the events that, on a given run, (i) the derivation d is valid, (ii) the reduction $\rho(d)$ is faithful, and (iii) the verifier accepts. Let $\mathsf{Cert} = G \cap R \cap V$ be the event of a genuinely certified claim, and $p_G = \Pr(G)$, $p_R = \Pr(R)$, $p_V = \Pr(V)$. Then*

$$\Pr(\mathsf{Cert}) \;\leq\; \min\{p_G,\, p_R,\, p_V\}.$$

Proof. $\mathsf{Cert} \subseteq G$, $\mathsf{Cert} \subseteq R$ and $\mathsf{Cert} \subseteq V$; apply monotonicity of probability and take the minimum of the three bounds. □

Remark 1 (Consequences). Two practical readings follow. First, investment in Layer 3 has *zero* effect on $\Pr(\mathsf{Cert})$ once p_R is the binding constraint — which is the regime we argue the field is in. Second, answer-level accuracy A is not an estimator of $\Pr(\mathsf{Cert})$: A counts runs whose final answer matches a reference, including runs with invalid reasoning. The difference $A - \Pr(\mathsf{Cert})$ is what we call the *tier gap*, and it is empirically measurable — on GSM8K it is witnessed by the 3.4% reasoning-error rate among final-answer-correct solutions reported by Uesato et al. [17].

4 The Semantic Reduction Bottleneck

4.1 Why Layer 2

Layer 1 fails loudly: a wrong answer is detectable by grading. Layer 3 fails informatively: a kernel reports the goal state, the failing tactic, the missing hypothesis. Layer 2 fails *silently*. Nothing in the generation layer checks it (the prose was never checkable), and nothing in the verification layer can see it (the kernel only ever sees Φ). Its failure modes are therefore invisible to both neighbours and appear downstream as a puzzling "the proof checked but the result is not what I meant".

The literature supports placing the bottleneck here. Autoformalisation is treated as a research problem in its own right, with dedicated benchmarks and baselines [1]. Premise selection over a library of hundreds of thousands of declarations was identified as a key bottleneck and addressed by retrieval [21]. Large formal efforts autoformalise tens of millions of statements to get enough training signal [8], and even at IMO 2024 the competition statements were manually translated into formal language before the provers ran [8]. PutnamBench reports that established provers solve only a handful of its problems [16] — consistent with a pipeline in which getting to a correct formal statement is itself a large part of the difficulty.

4.2 A taxonomy of reduction failures

Table 1 collects seven failure modes. The first three were identified in the challenge outline for this work (ambiguity, missing typing, non-invertibility); we add four that arise from practice with real libraries.

The rightmost column is the practical punchline: four of the seven modes (F3, F6, F7, and partly F5) produce *no* signal in the verification layer. A pipeline can therefore be perfectly sound at Layer 3 and still certify the wrong thing.

5 The Proposed Framework

5.1 Design of the typed IR

We specify Layer 2 as a typed intermediate representation IR with four components: (i) a *type system* in which every quantified variable has a declared type; (ii) an *operational semantics* fixing the meaning of each primitive, including totalised conventions (what is $x/0$? what is $\sqrt{\cdot}$ on negatives?); (iii) a *constraint system* carrying side conditions as first-class hypotheses rather than prose; and (iv) a *compositional structure* so that partial reductions can be checked independently. The theoretical warrant is the propositions-as-types correspondence [18]: with proofs as programs, a reduction is a compilation step and its correctness is a typing claim, not a matter of taste.

Table 1: A taxonomy of semantic-reduction failure modes. "Layer-3 signal" indicates whether a verifier would notice the failure on its own.

	Mode	Description	Detection signal	Layer-3 signal
F1	Lexical / symbolic ambiguity	One symbol carries two readings (e.g. $-$ as binary vs. unary; $\|G\|$ as order vs. cardinality; $\mathbb{N}$ including 0 or not).	Type-checking; explicit symbol table	Yes
F2	Missing typing	Quantified variables have no declared type or domain; implicit coercions.	Elaboration error	Yes
F3	Non-invertibility	The map is many-to-one or convention-dependent; rendering back to prose is not the inverse of reduction.	Round-trip comparison	No
F4	Premise / library mismatch	The invoked lemma exists but is a different result (same name, different generality) or is unavailable.	Premise resolution to named declarations	Partly
F5	Hidden side conditions	Steps presuppose $x \neq 0$, convergence, compactness, non-emptiness, or a choice of branch.	Goal-state inspection; degenerate-case tests	Partly
F6	Non-computational steps	Existence / choice / diagonal arguments with no computational witness.	Manual inspection	No
F7	Round-trip drift	The rendered natural language no longer states c; readers certify the rendering, not the proof.	Independent equivalence judgement	No

5.2 Pipeline and bidirectional alignment

The pipeline is generation → reduction → verification → rendering, with a mandatory *round-trip*: informalise Φ back to natural language and require an independent judgement that the result is equivalent to c (Figure 1). Round-tripping is not decoration — it is the only check that catches F3 and F7, which are otherwise undetectable.

5.3 The Reliability Ladder

Table 2 is the paper's main operational artefact. Three properties are worth stressing. *Monotonicity*: L4 does not replace L3, it contains it; a serious L4 result should still report its L2 spot-checks. *Separation*: L4 is the only tier at which Prop. 1 is defeated, because it is the only tier that explicitly attests $[\![\rho(d)]\!] = [\![c]\!]$. *Cost honesty*: the rightmost column is why nobody lives at L4 by default — the strongest published formal results required human formalisation and, in the IMO case, multi-day computation [8].

5.4 The Reduction Verification Checklist

Table 3 is meant to be used as a gate: an output that fails any of C1–C10 should not be reported above L2.

C12 deserves comment. Reporting "we proved 60% of the benchmark" is compatible with "we attempted 100 reductions and 40 failed silently". Because Layer 2 fails silently, a denominator taken over successful reductions hides exactly the failures this paper is about.

Table 2: The Reliability Ladder. Tiers are ordinal: L4 subsumes L3, which subsumes L2, and so on. Each tier states what a reader is entitled to conclude.

Tier	What was done	What it licenses	Residual risk	Typical cost
L0	Model produced prose	Nothing; output is a hypothesis, not a result	Unbounded	1×
L1	Self-consistency, majority voting, learned reranking [3]	The output is not idiosyncratic to one sample	Correlated errors across samples	n×
L2	External computation: calculator, CAS, numeric spot-checks [13, 2]	Numeric agreement on the tested instances	Degenerate and non-computable cases missed	2–5×
L3	Step-level process verification by a learned verifier or human step labels [17, 11]	Each step was endorsed; errors are localised	The endorser is itself unverified	10–100×
L4	Machine-checked proof in a trusted kernel *and* attested-faithful reduction [5, 8]	The formal statement holds in T; with a faithful reduction, so does c	Kernel/axiom trust; reduction faithfulness	100–10^4×

5.5 A fidelity metric (proposed)

Reduction Fidelity Index. Over m checkpoints, let $r_i \in \{0, 1\}$ indicate that checkpoint i passed round-trip equivalence (C7) and its spot-checks (C8–C9). Define

$$\mathrm{RFI} \;=\; \frac{1}{m}\sum_{i=1}^{m} r_i.$$

We also report, separately and never as a substitute: the reduction *acceptance rate* a (fraction of attempts producing a well-typed Φ), the *certified rate* $r = \Pr(\mathsf{Cert})$, answer-level accuracy A, and the tier gap $A - r$. RFI is a proposed metric, not a validated one; it requires human adjudication and we do not claim it is the right weighting.

6 Re-analysis: The Tier Gap in Published Results

Table 4 re-reads published results through the ladder. The pattern is consistent: the tier that the evidence actually supports is below the tier that the surrounding narrative invites. We intend this as a methodological observation, not as a criticism of the cited work, all of which is careful about its own claims.

Two entries deserve emphasis. *AlphaProof* is the clearest L4 result in the table, and it is precisely the case where the reduction layer was human-staffed: the IMO statements were translated into formal language by people before the system ran [8]. That is not a criticism of the system; it is a measurement of how expensive Layer 2 currently is. *Process supervision* is the clearest measurement of the tier gap: the 3.4% residual reasoning error among correctly answered problems [17] is the gap made numeric.

Table 3: The twelve-item Reduction Verification Checklist (RVC). Items marked $*$ are the ones we find most often skipped in practice.

	Item	**Failure mode caught**
C1	Every quantified variable has an explicit declared type	F2
C2	Side conditions ($x \neq 0$, convergence, compactness, non-emptiness) are explicit hypotheses	F5
C3	Every symbol has one declared meaning; overloaded notation disambiguated	F1
C4	Every invoked lemma resolves to a named declaration in the target library	F4
C5$*$	Library and prover versions are pinned and reported	F4
C6$*$	Both directions checked: formal statement $\Rightarrow$ informal claim, and conversely	F3, F7
C7	Round-trip: informalise($\rho(d)$) is judged equivalent to c by an independent reader	F3, F7
C8$*$	Numeric spot-checks on randomly instantiated instances	F5
C9	Degenerate cases tested explicitly (0, 1, $\emptyset$, boundary, ∞)	F5
C10	Each step is kernel-checked or explicitly flagged as assumed	F6
C11$*$	Provenance recorded: which parts are model-generated vs. human-written	F7
C12$*$	Acceptance rate reported over *attempted* reductions, not only successful ones	all (reporting bias)

7 Worked Case Study (Constructed)

We now exhibit the reduction gap on an elementary example. **This example is constructed for exposition and involves no measurement**; its mathematics is elementary and can be checked by hand.

The claim. Consider

$$c: \qquad \text{“For every real number } x, \quad \frac{x^2-1}{x-1} = x+1.\text{”}$$

A plausible generation (L0). A language model asked to verify c typically produces something like:

> Factor the numerator: $x^2-1 = (x-1)(x+1)$. Then $\frac{(x-1)(x+1)}{x-1} = x+1$ after cancelling the common factor $(x-1)$. Hence the identity holds.

The prose is fluent, the algebra is familiar, and the conclusion is *false*: at $x = 1$ the left-hand side is not defined (and under the totalised-division convention used by most proof assistants it evaluates to 0, while the right-hand side evaluates to 2). The missing step is the side condition $x \neq 1$ — failure mode F5.

Behaviour at each tier.

- **L0.** Passes. Nothing in the output signals a problem.

Table 4: Re-analysis of published systems through the Reliability Ladder. "Evidenced" is the tier the reported artefact supports; "Implied" is the tier a reader commonly takes away. All numbers are as reported in the cited source (see Appendix A).

System	Artefact	Headline result	Evid.	Impl.	Why the gap
Minerva [10]	MATH, STEM	SOTA without external tools	L0–L1	L2	Majority voting; no external check
Verifiers [3]	GSM8K	Reranking improves accuracy	L1	L2	The verifier is a learned model, itself ungraded
Process supervision [17]	GSM8K	16.8→12.7% answer error; 14.0→3.4% reasoning error	L2	L3	Step labels are human; no kernel
PRM / PRM800K [11]	MATH subset	78% under best-of-n	L2	L3	Reward model, not a proof checker
Toolformer, PoT [13, 2]	Arithmetic, QA	Improved zero-shot accuracy	L2	L2	Computation externalised; reasoning not
AlphaGeometry [15]	30 olympiad geometry	25 solved	L3	L4	Symbolic engine checks, but in a domain-specific language
AlphaProof [8]	IMO 2024	4/6 problems, 28/42 points	L4	L4	Kernel-checked, but statements formalised by humans; multi-day compute
Neural provers [22, 1, 16]	miniF2F, ProofNet, PutnamBench	Low single-digit to modest pass rates	L4	L4	Genuine L4 where solved; coverage is the limit
FunSearch [12]	Cap set, bin packing	New constructions beating known bounds	L3	L4	Evaluator checks programs; insight is external
Frontier models [6]	FrontierMath	Under 2% solved	L0–L1	L2	Auto-verifiable answers; reasoning unexamined

- **L1.** Passes. Independent samples produce the same cancellation argument, so majority voting and reranking are *confidently* wrong: consistency is not correctness.
- **L2.** Passes with high probability. A numeric spot-check samples x from $\mathbb{R}$; the single offending point $x = 1$ has Lebesgue measure zero, so random testing will not find it. This is the instructive part: *sampling-based checking is structurally blind to degeneracies*, which is exactly why C9 (explicit degenerate-case tests) is on the checklist and not subsumed by C8.
- **L3.** Flags it. A step-level check of "cancel $(x - 1)$" demands the precondition $x - 1 \neq 0$, which is absent.
- **L4.** Rejects it, and in doing so reveals a second phenomenon.

The reduction is not unique (F3). Formalising c requires choosing a convention for division at zero. In Lean's field convention division is totalised with $0^{-1} = 0$, so the unconditioned statement is simply *false* and the kernel rejects it; the faithful reduction must carry the hypothesis. Schematically:

```
-- UNFAITHFUL reduction: no side condition. This statement is FALSE
   at x = 1.
theorem bad (x : R) : (x^2 - 1) / (x - 1) = x + 1 := by
  ring_nf           -- fails: the goal is false at x = 1

-- FAITHFUL reduction: the hidden side condition is now a hypothesis.
```

```
theorem good (x : R) (hx : x <> 1) : (x^2 - 1) / (x - 1) = x + 1 :=
   by
  have h : x - 1 <> 0 := sub_ne_zero.mpr hx
  field_simp [h]
  ring
```

Listing 1: Illustrative Lean fragment. Not executed or type-checked in this work; shown to make the side condition explicit.

The same English sentence therefore reduces to two formal statements with *different truth values*, depending on a convention the prose never mentions. That is non-invertibility in the sense of F3, and it is invisible to Layer 3: the kernel is impeccably correct in both cases. It is only visible if someone performs C6 (check both directions) and C7 (round-trip).

What the case study shows. Three things, in order of importance: (1) tiers L0–L2 can all pass on a false claim, and L1 can pass *confidently*; (2) the failure is undetectable by random numeric testing because the failure set has measure zero; and (3) the reduction step is convention-dependent, so "the formal statement" is not uniquely determined by the English — which is precisely the situation Proposition 1 describes.

8 A Pre-specified Protocol for Measuring Reduction Fidelity

We close the methodological loop by pre-specifying how the framework would be tested. **This protocol is not executed in this paper**; we give it so that the framework is falsifiable and so that the required sample size is visible in advance.

Corpus. Use ProofNet [1] as the primary instrument: its 371 items each pair a natural-language statement, a natural-language proof, and a reference formal statement, which supplies a *gold* target against which reduction fidelity can be scored rather than merely asserted. miniF2F [22] (488 statements) and PutnamBench [16] provide harder and broader secondary corpora.

Procedure. For each item i: (1) a generator produces an informal derivation d_i; (2) a reduction module (model plus deterministic compiler) produces Φ_i; (3) fidelity is adjudicated by comparing Φ_i to the gold formal statement, using definitional equality where available and two independent human adjudicators otherwise; (4) a kernel check is attempted and the goal state recorded; (5) checklist C8–C9 spot-checks are run.

Reported quantities. Reduction acceptance rate a; RFI; answer-level accuracy A; certified rate r; tier gap $A - r$; and the conditional gap given document-style fluency, $A - r$ restricted to outputs an expert rates as plausible. The last quantity is the one that matters most for users: it is the probability that a plausible-looking answer is wrong.

Sample size. For a two-proportion comparison at $\alpha = 0.05$, power 0.80, with $p_1 = 0.5$, $p_2 = 0.4$ and $\delta = 0.10$,

$$n \approx \frac{(z_{0.975} + z_{0.80})^2 \left(p_1(1-p_1) + p_2(1-p_2)\right)}{\delta^2} \approx 385 \text{ items.}$$

ProofNet's 371 items are therefore adequate for detecting a ten-point gap and *under-powered* for smaller ones — which is worth stating in advance, because a null result on 371 items would not license the conclusion that reduction is faithful.

Falsification. The framework's central empirical claim is that p_R is the binding constraint in Proposition 2. It would be refuted by a study in which, on a non-trivial corpus, reduction acceptance a and fidelity are both high while certified rate r remains low — i.e. the binding constraint sits at Layer 1 or Layer 3 after all.

9 Threats to Validity and Limitations

- **No new empirical results.** Section 7 is constructed; Section 8 is unexecuted. The framework's empirical claim is stated as a hypothesis with a falsification condition, not as a finding.
- **Faithfulness has no formal ground in general.** $[\![c]\!]$ presupposes a semantics for informal mathematical language, which does not exist in general. We operationalise faithfulness against gold formalisations and human adjudication (Section 8); this makes the notion usable but leaves it partly judgement-dependent.
- **The ladder is ordinal.** L4 is not "four times L1". Tier assignments in Table 4 involve interpretation, and reasonable readers may disagree at the margin; we have tried to state the basis for each assignment in the final column.
- **Kernel and library trust.** L4 assumes the proof checker, its axiomatisation, and the library are correct. A large trusted base [14] is a real assumption, not a formality.
- **Cost.** L4 is expensive in human effort and compute [8]. Our argument is that the ladder should be reported honestly, not that everyone should live at L4.
- **A moving field.** All reported numbers are as of the cited publications and were re-checked on 2026-09-25; several benchmarks are actively being saturated.

10 Conclusion

Reliability in AI mathematics is not a property of a model; it is a property of a pipeline, and pipelines have three layers. We argued that the middle one — semantic reduction, the map from informal argument to checkable object — is where reliability is most often lost, because it is the only layer whose failures are invisible to both of its neighbours. Proposition 1 shows that a sound verifier certifies the reduced object and not the user's claim; Proposition 2 shows that effort spent on generation or verification is wasted while reduction is the binding constraint. We contributed a Reliability Ladder and a Reduction Verification Checklist so that this can be audited rather than asserted, re-read the recent literature through them to expose a systematic tier gap, and pre-specified a protocol for measuring reduction fidelity.

The practical recommendation is modest and cheap: *publish the tier*. A result reported as "78% on a MATH subset" and the same result reported as "78% at L2, with no kernel-checked proof and no round-trip verification" are different scientific objects. Making the tier explicit costs a sentence and removes most of the ambiguity that this paper has tried to describe.

A Provenance and Verification Record

Every external quantitative claim in this paper was checked against its source on 2026-09-25. This appendix records the check so that the claims can be audited independently. *Verification method* describes how the claim was confirmed: **P** = publisher/primary page, **A** = arXiv abstract page, **S** = authoritative secondary index (ADS, PubMed, DBLP, or a publisher-hosted citation record).

Claim as used in this paper	Source	Method	Outcome	Status
AlphaProof: 4/6 IMO 2024 problems, 28/42 points, silver-medal equivalent	[8]	P + A (PMC full text)	Confirmed	OK
IMO 2024 statements were manually translated into formal language for the system	[8]	P (press release, PMC)	Confirmed	OK
AlphaProof trained on ∼80M autoformalised problems	[8]	P (press release)	Confirmed	OK
AlphaGeometry solves 25 of 30 olympiad geometry problems	[15]	P	Confirmed	OK
miniF2F: 488 problem statements	[22]	A + P (ICLR)	Confirmed	OK
ProofNet: 371 examples	[1]	A + P (repo)	Confirmed; journal version *not* confirmed	Cited as preprint
PutnamBench: 1692 formalisations, 640 theorems; provers solve a handful	[16]	A	Confirmed	OK
LeanDojo: 98,734 theorems; premise retrieval	[21]	A + P (NeurIPS)	Confirmed	OK
MATH: 12,500 problems	[7]	A + P	Confirmed	OK
GSM8K: 8.5K problems	[3]	A (ADS)	Confirmed	OK
16.8→12.7% answer error; 14.0→3.4% reasoning error	[17]	A (arXiv PDF)	Confirmed verbatim	OK
PRM: 78% of a MATH subset; PRM800K labels	[11]	A (ADS + arXiv)	Confirmed	OK
FrontierMath: models solve under 2%	[6]	A	Confirmed	OK
FunSearch: new cap-set constructions; Nature 625:468–475	[12]	P	Confirmed	OK
Lean 4: CADE 28, LNCS 12699, pp. 625–635	[5]	P (Springer/KIT)	Confirmed	OK
mathlib: CPP 2020, pp. 367–381	[14]	S (cited record in [5])	Confirmed	OK
Toolformer: NeurIPS 36, pp. 68539–68551	[13]	S (Scholar/NeurIPS)	Confirmed	OK
Lean's totalised division ($0^{-1} = 0$) makes the unconditioned identity false at $x = 1$	[5, 14]	Derived by hand	Convention-dependent; stated as such in §7	Qualified

Declared corrections. Two items were adjusted during verification and are recorded here rather than silently fixed:

(1) ProofNet is cited as an arXiv preprint. A journal version (TMLR) was suggested during drafting but could not be confirmed, so no journal venue is claimed.

(2) The AlphaProof Nature paper is cited by DOI only. Volume and page numbers were not confirmed at the time of writing and have not been invented.

AI-assisted provenance. Parts of this manuscript were drafted with the assistance of a language model. All literature claims were subsequently verified against sources by the author; the per-claim record is the table above, and the day-by-day collaboration record — including cases where the model produced an incorrect claim that was caught in verification — is shipped with this manuscript as the accompanying AI collaboration log and after-action review.

References

[1] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics, 2023. 371 examples; cited as preprint (journal version unconfirmed).

[2] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, 2022. Published in Transactions on Machine Learning Research (TMLR), 2023.

[3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. Introduces GSM8K (8.5K problems) and best-of-n verification.

[4] Alex Davies, Petar Velickovic, Lars Buesing, Sam Blackwell, Daniel Zheng, Nenad Tomasev, Richard Tanburn, Peter Battaglia, Charles Blundell, Andras Juhasz, Marc Lackenby, Geordie Williamson, Demis Hassabis, and Pushmeet Kohli. Advancing mathematics by guiding human intuition with ai. *Nature*, 600(7887):70–74, 2021.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[6] Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, Caroline Falkman Olsson, Jean-Stanislas Denain, Anson Ho, Emily de Oliveira Santos, Olli Jarviniemi, Matthew Barnett, Robert Sandler, Jaime Sevilla, Qiuyu Ren, Elizabeth Pratt, Lionel Levine, Grant Barkley, Natalie Stewart, Bogdan Grechuk, Tetiana Grechuk, Shreepranav Varma Enugandla, and Mark Wildon. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai, 2024. Current models solve under 2

[7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. Datasets and Benchmarks Track; 12,500 competition problems; arXiv:2103.03874.

[8] Thomas Hubert, Rishi Mehta, Laurent Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. AlphaProof; IMO 2024 silver-medal equivalent (28/42 points).

[9] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):248:1–248:38, 2023.

[10] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models, 2022. Minerva; no external tools.

[11] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. PRM800K; 78

[12] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. FunSearch.

[13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 68539–68551, 2023. arXiv:2302.04761.

[14] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[15] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

[16] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. 1692 formalizations of 640 theorems; Lean 4, Isabelle and Coq.

[17] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. 16.8

[18] Philip Wadler. Propositions as types. *Communications of the ACM*, 58(12):75–84, 2015.

[19] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903.

[20] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, et al. Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search, 2024.

[21] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Datasets and Benchmarks Track; 98,734 theorems extracted from mathlib.

[22] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2109.00110; 488 problem statements.